Appendix B Solutions

GB-Q01

Symbol	Address
main	`0x00400000`
loop	`0x00400004`
done	`0x00400010`
func	`0x00400018`
table	`0x10010000`
count	`0x10010004`
str	`0x10010008`

The branch is at `0x00400008`; its base is `0x0040000c`. Thus $(0x00400004 - 0x0040000c)/4 = -2$, encoded `0xfffe`.

Target fields are `(target >> 2) & 0x03ffffff`: `j done` uses `0x00100004`; `jal func` uses `0x00100006`. These calls are hypothetical, so they do not shift labels. `.align 2` provides four-byte alignment. The two words occupy eight bytes; `Hi` plus zero occupies three, with final NUL at `0x1001000a`.

GB-Q02

The target field is $0x00402000 \gg 2 = 0x00100800$. The complete word is $(3 \ll 26) | 0x00100800 = 0x0c100800$. The call-site PC+4 and target share the required upper nibble.

For ORI, high is `0x1234` and low `0x5678`. Generally,

$$H_{ori} = (V \gg 16)\ \&\ 0xffff, \quad H_{addiu} = ((V + 0x8000) \gg 16)\ \&\ 0xffff.$$

ADDIU sign-extends the low field; a low half with bit 15 set therefore needs the adjusted high half. For `0x1234abcd`, ORI uses high `0x1234` and low `0xabcd`. ADDIU uses high `0x1235` and signed low −21555. Both reconstruct the same 32-bit value modulo 2^{32}. Mixing the adjusted high with ORI gives the wrong value.

The field can initially hold a placeholder or implicit addend. Metadata tells the linker the symbol, site, type, and how to combine them. It distinguishes a literal constant from an address needing relocation. The terminology follows the MIPS32 instruction reference for LUI, ORI, and ADDIU, also cited by the Chinese baseline.

Appendix B Solutions (continued)

GB-Q03

P text begins at `0x00400000`; Q begins at $0x00400000 + 192 = 0x004000c0$. P data begins at `0x10010000`; Q follows at `0x10010010`.

The latter is already eight-byte aligned. Q ends exclusively at $0x10010010 + 24 = 0x10010028$; its last occupied byte is one less.

Two strong global definitions normally cause a duplicate-symbol error. Weak or common rules can permit a specified resolution, and local names can coexist in separate objects. The linker must follow binding rules rather than silently choose an arbitrary semantic result.

GB-Q04

Arguments 3, 2, 1, and 0 create four simultaneous frames, totaling 64 bytes. Within each, addresses are current `$sp+12` for saved `$ra` and `$sp+8` for saved `$a0`; each recursion changes the base by sixteen.

Return proceeds from inner to outer. A conventional epilogue loads saved state before adding sixteen to `$sp`. If the increment comes first, old offset twelve addresses a different location; an adjusted offset such as −4 or a saved old base can still reach the old slot. The old frame is not literally unreachable.

Ordinary ABI rules preserve selected registers across calls. Interrupt entry must preserve whatever interrupted state its handler can otherwise destroy, along with PC/status information. Nested handlers need separate safe storage, bounded stack use, and reentrancy; an exception is not an ordinary `jal` call.

GB-Q05

Big-endian bytes are `de ad be ef`; little-endian bytes are `ef be ad de`. Address `0x102` is not divisible by four, so strict `lw` raises an address/alignment fault rather than a normal word load.

The separately stored big-endian halfword has bytes `be ef`; `lhu` returns `0x0000beef` by zero extension.

Native dumps reflect host order, padding, alignment, and compiler layout. A wire format independently specifies widths and byte order and requires explicit serialization/validation.

Appendix B Solutions (continued)

GB-Q06

The teaching service number is in `$v0`, with integer argument in `$a0`. Some returning services use `$v0` or another specified register, but print-integer itself has no designated numerical return result.

Given BD=0 and EPC naming the syscall, completing its service means continuing with the next instruction, EPC+4. This is a handler policy under the stated model, not proof that the simulator internally runs this handler. If a simulator already advances execution, do not advance twice.

For BD=1, EPC instead names the preceding branch and recovery must respect its continuation. For overflow, the faulting result has not validly committed. Skipping it can leave an incorrect destination. Termination, emulation of a defined alternative, or context repair is required before deciding whether to retry or continue.

A handler that clobbers application registers changes the interrupted program. Nested entry can also destroy the outer handler's temporary state. Save/restore, stack capacity, masks, and reentrancy are necessary parts of the recovery contract.

GB-Q07

```
move $s0,$s1 -> addu $s0,$s1,$zero
la $t0,label -> lui $t0,raw_upper16(label)
                ori $t0,$t0,raw_lower16(label)
```

This ORI expansion uses raw halves, without signed-low adjustment. A different assembler can use ADDIU with its adjusted high relocation, a small immediate sequence, or GOT/PC-related addressing where supported. Address range and PIC mode determine the actual expansion.

SPIM print-string uses `$v0=4` and `$a0` pointing to the terminated bytes. Print-integer uses `$v0=1` and `$a0` holding the integer. Exit uses `$v0=10`. Each service is requested with `syscall` under that simulator ABI.

Pseudo expansion, directives, padding, relaxation, and delay-slot scheduling mean source-line count does not establish machine-byte count. Inspect the actual listing and linked image.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Appendix B Solutions (continued)

GB-Q08

P.text ends exclusively at 0x00401024; align Q to 0x00401030, padding 12 bytes. Q ends 0x00401044. P.data ends 0x10010005; Q.data 0x10010008 requires 3 padding bytes and ends 0x1001000c. Last-byte addresses are exclusive ends minus one.

Foo=0x00401034, item=0x10010008. Jal site 0x00401008 gives field 0x0010040d, word 0x0c10040d, and ra 0x00401010. Its slot at site+4 executes first.

For item 0x10010008, high 0x1001, low 0x0008. For 0x1001fffc, low 0xfffc means −4 and adjusted high 0x1002; 0x10020000−4 restores the target. Unadjusted 0x1001 would give 0x1000fffc, off by 0x10000.

The field encodes target bits 27:2 only; CPU supplies PC+4's high nibble. Another region is unrepresentable. Report a range error or use a legal veneer/indirect sequence rather than silent truncation.

GB-Q09

After main, U={f}. libA extracts A1, defines f, and introduces g, so U={g}; A2 is not needed yet. libB extracts B1 and introduces h, so U={h}. The scan has passed libA and fails without revisiting it.

A second libA scan extracts A2 and empties U. Repetition or group/rescan options handle such dependencies according to the linker. If A1/A2 form one member, needing f extracts the whole member, including h. B1's later reference is already resolved, and the original order succeeds. Extraction granularity is an object member, not individual symbol bytes.

GB-Q10

n	sp	Saved n	Saved ra	Returned v0
2	0x1ff0	0x1ff8	0x1ffc	3
1	0x1fe0	0x1fe8	0x1fec	1
0	0x1fd0	0x1fd8	0x1fdc	0

Maximum 48 bytes; final sp 0x2000. t0 is caller-saved. If outer n=2 loads t0=2 before the call but child n 1 leaves t0=1, the outer computes 1+1=2 instead of 3. Reloading saved n from the frame after the call avoids this clobber.

Appendix B Solutions (continued)

A jal slot runs before entering the child. Its v0 is not yet sum_to(n−1), and the child later overwrites it. Premature addition reads at the wrong time and does not preserve its result.

Require nonnegative n, representable n(n+1)/2, valid alignment, and enough stack. Base n=0 returns 0 and restores state. Inductively, the child returns (n−1) n/2 and the parent's sp; the parent reloads its saved n, adds it, and restores its ra/sp. Nonoverlapping frames preserve live values; decreasing n to 0 proves termination. A finite trace does not replace induction.

GB-Q11

BD=1, EPC=0x00400040. After mapping repair, retry must preserve branch/slot semantics. EPC+4 alone reaches 0x00400044 without the branch decision. Handler state must follow the specific ISA/fault policy rather than blindly skipping faults.

The completed BD=0 syscall instead continues at 0x00400084 under the stated manual-advance policy. Its service must not repeat, while the faulting load still requires completion. Avoid an extra advance if a different simulator already provides one.

Control is read three times, data once, and v0=0x141&0xff=0x41. A data load in the branch slot also executes when ready 0 and can consume invalid data or change state. It violates read-only-after-ready and consume-once behavior. The nop preserves this protocol.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Appendix B Solutions (continued)

Verification code appendix: assembly_toolchain.py

```
"""Small assembler, relocation, ABI, and exception teaching model."""

from __future__ import annotations

import ast
import re


LABEL = re.compile(r"^\s*([A-Za-z_.$][\w.$]*):\s*(.*)$")


def first_pass_symbols(source: str, text_base: int = 0x00400000, data_base: int = 0x10010000)
-> dict[str, int]:
    """Assign labels for explicit-align data and single-word instructions."""
    symbols: dict[str, int] = {}
    section = "text"
    locations = {"text": text_base, "data": data_base}
    for raw in source.splitlines():
        line = raw.split("#", 1)[0].strip()
        if not line:
            continue
        location = locations[section]
        label_match = LABEL.match(line)
        if label_match:
            symbols[label_match.group(1)] = location
            line = label_match.group(2).strip()
            if not line:
                continue
        parts = line.split(None, 1)
        directive = parts[0].lower()
        if directive == ".text":
            section = "text"
            continue
        if directive == ".data":
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Appendix B Solutions (continued)

```
            section = "data"
            continue
        if directive == ".align":
            exponent = int(parts[1])
            alignment = 1 << exponent
            location = (location + alignment - 1) & -alignment
        elif directive == ".word":
            location += 4
        elif directive == ".asciiz":
            literal = ast.literal_eval(parts[1].strip())
            location += len(literal.encode("utf-8")) + 1
        elif section == "text":
            location += 4
        else:
            raise ValueError(f"unsupported data directive: {line}")
        locations[section] = location
    return symbols


def j_target_field(target: int) -> int:
    return (target >> 2) & 0x03FFFFFF


def jal_word(target: int) -> int:
    return (3 << 26) | j_target_field(target)


def branch_word_index(pc: int, target: int) -> int:
    delta = target - (pc + 4)
    if delta % 4:
        raise ValueError("unaligned branch target")
    value = delta // 4
    if not -32768 <= value <= 32767:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Appendix B Solutions (continued)

```
        raise ValueError("branch target outside 16-bit range")
    return value & 0xFFFF


def load_addiu_constant_halves(value: int) -> tuple[int, int]:
    value &= 0xFFFFFFFF
    low = value & 0xFFFF
    high = ((value + 0x8000) >> 16) & 0xFFFF
    return high, low


def main() -> None:
    source = """
    .text
    main: addiu $t0, $zero, 0
    loop: addiu $t0, $t0, 1
          beq   $t0, $a0, loop
          nop
    done: jr    $ra
          nop
    func: addiu $v0, $zero, 7
          jr    $ra
          nop
    .data
    .align 2
    table: .word 0xdeadbeef
    count: .word 3
    str:   .asciiz "Hi"
    """
    symbols = first_pass_symbols(source)
    assert symbols == {
        "main": 0x00400000,
        "loop": 0x00400004,
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Appendix B Solutions (continued)

```
        "done": 0x00400010,
        "func": 0x00400018,
        "table": 0x10010000,
        "count": 0x10010004,
        "str": 0x10010008,
    }
    assert branch_word_index(0x00400008, symbols["loop"]) == 0xFFFE
    assert j_target_field(symbols["done"]) == 0x00100004
    assert jal_word(0x00402000) == 0x0C100800
    assert j_target_field(symbols["func"]) == 0x00100006
    assert first_pass_symbols('.text\na: nop\n.data\nx: .word 1\n.text\nb: nop') == {"a":
0x00400000, "x": 0x10010000, "b": 0x00400004}
    for value in (0x12345678, 0x1234ABCD, 0xFFFF8000):
        high, low = load_addiu_constant_halves(value)
        signed_low = low if low < 0x8000 else low - 0x10000
        assert ((high << 16) + signed_low) & 0xFFFFFFFF == value
        assert (((value >> 16) << 16) | low) == value
    assert 4 * 16 == 64
    assert (0x00400000 + 192) == 0x004000C0
    assert (0x10010010 + 24) == 0x10010028

    print("symbols:", " ".join(f"{name}=0x{value:08x}" for name, value in symbols.items()))
    print(f"beq immediate: 0x{branch_word_index(0x00400008, symbols['loop']):04x}")
    print(f"jal word: 0x{jal_word(0x00402000):08x}")
    print("lui/addiu adjusted halves:", tuple(hex(x) for x in
load_addiu_constant_halves(0x1234ABCD)))
    print("sum_to(3) stack bytes:", 4 * 16)
    print("toolchain checks: PASS")


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study